From Single-Request Inference to Concurrent Serving

Throughput–Latency Tradeoffs for Qwen3.8-27B on an H100

Mihal Dimo

github.com/mhdimo/qwen38-h100-lab

1 October 2026

Abstract

We evaluate an isolated concurrent-inference implementation for Qwen3.8-27B-FP8 on one NVIDIA H100 80GB GPU. Inspection of the existing deployment reveals a one-sequence engine limit, a 2 GiB cache budget, and an application lock that rejects overlapping inference. Our implementation uses vLLM AsyncLLM with a 64-sequence limit, 16 GiB cache, and CUDA-graph capture sizes for concurrent decoding. With 32 concurrent synthetic requests, the 16,384-token scheduling budget increases decode-heavy aggregate output throughput from 74.37 to 1,645.66 tokens/s (22.13×), and mixed throughput from 71.70 to 1,003.64 tokens/s (14.00×). Prefill-heavy input throughput rises by 51.79%. These gains trade higher memory use and per-request token latency for much less queueing. Single-request first-token latency regresses. The comparison is between matched engine configurations under an asynchronous harness, not an HTTP production benchmark or a novel GPU-kernel implementation. Three configurations completed; later experiments were cancelled.

1 System question and contribution

The objective is maximum aggregate throughput for many concurrent users, rather than minimum latency for one request. vLLM exposes batching and memory-management mechanisms for this purpose [1]. However, the existing application's admission policy prevents concurrent requests from reaching the engine. Optimizing a GPU kernel alone cannot remove that constraint.

The contribution of this report is a reproducible local benchmark implementation, a measured comparison of engine policies, and a documented integration boundary. Across measured configurations we preserve the checkpoint, numerical formats, runtime and context limit. We do not claim a new attention algorithm, custom CUDA kernel, or universal fastest configuration.

Table 1: Deployed worker versus the proposed implementation.

Property	Current deployed worker	Proposed isolated implementation
Request path	Synchronous `LLM.generate`; application busy lock	Bounded asynchronous request pool feeding `AsyncLLM`
Sequence capacity	1	64
Fixed cache budget	2 GiB	16 GiB
Prefill token budget	2,048 in reproduced baseline	8,192 or 16,384
Compilation / graphs	Enabled; prefill-oriented captures through 2,048	Enabled; captures through batch size 64
Deployment status	Existing service	Separate lab; not integrated into service

2 Experimental method

2.1 Hardware, model and software

All inference ran on one isolated NVIDIA H100 80GB HBM3 with an observed 700 W power limit and driver 610.57.04. The software stack was Python 3.14.6, vLLM 0.26.0, PyTorch 2.11.0, Triton 3.6.0, Transformers 5.16.1 and FlashInfer 0.6.14. The cached checkpoint was `Qwen/Qwen3.8-27B-FP8`.
Pinned revision: `017b9c7af6b5689d5dd426a76e0bc077eb5ca20a`. Qwen3.8-27B combines Gated DeltaNet and full-attention layers in the Qwen3.5 architecture [2]. The native FP8 checkpoint and automatic BF16 cache format were unchanged; maximum context length was 10,240 tokens.

Resolved logs identify FP8 DeepGEMM, FlashAttention 3, FlashInfer GDN prefill, asynchronous engine scheduling, and norm/activation quantization fusions. This is evidence of selected implementations, not a measurement of their individual time shares.

2.2 Baseline construction and fairness

Every measured configuration uses the same direct AsyncLLM client. The baseline reproduces the deployed engine's one-sequence limit, 2 GiB cache and graph policy. At client concurrency 32, its engine queues requests. The actual deployed wrapper instead raises a busy error when another inference is running. Therefore the baseline is an *engine-capacity proxy*, not a measurement of accepted HTTP traffic, network latency or production error rate.

The first candidate changes sequence capacity, cache capacity and graph capture policy together. Its gain must be attributed to that combined configuration. The 8,192-versus-16,384 comparison changes only the scheduled-token budget. Graph mode is `FULL_AND_PIECEWISE`; candidate capture sizes are {1,2,4,8,16,24,32,48,64}. The baseline uses {1,2,128,160,256,512,1024,2048}.

Table 2: Fixed-length synthetic workloads. Each configuration has two repetitions.

Workload	Input tokens	Output tokens	Concurrency	Requests / repetition
Single-request decode	128	128	1	8
Prefill-heavy	4,096	1	32	64
Decode-heavy	128	128	32	64
Mixed	1,024	128	32	64

Prompts are seeded, unique token-ID sequences, shared across candidates. Fixed output length is enforced with `ignore_eos`, `min_tokens` and `max_tokens`. Warmup uses separate prompts and the supported concurrency shape, followed by prefix-cache reset. All reported measured requests have zero cached prompt tokens. These inputs exercise tensor shapes, not representative language or vision semantics. Each complete configuration contributes 400 measured requests; the three complete configurations contribute 1,200.

2.3 Metrics and repetition handling

Let T be wall-clock duration from the first client dispatch until all requests complete. For request i, let s_i, f_i and e_i denote submission, first observed output, and completion; P_i and O_i are token counts. We report

$$R_{\text{out}} = \frac{\sum_i O_i}{T}, \quad R_{\text{in}} = \frac{\sum_i P_i}{T}, \quad \text{TTFT}_i = f_i - s_i, \quad \text{TPOT}_i = \frac{e_i - f_i}{O_i - 1}.$$

TPOT is undefined for one-token outputs. It is a client-observed request average, not an individual inter-token latency distribution. Repetition throughput pools token totals and wall times; p50/p95 pool per-request samples. Two repetitions support descriptive repeatability, not confidence intervals or significance tests.

3 Measured throughput

Table 3: Completed runs only. Output throughput unless the row says input.

Workload / metric	Baseline	8,192 budget	16,384 budget	16,384 / baseline
Single-request output tok/s	74.31	72.83	72.49	0.98×
Prefill input tok/s	11,919.10	17,683.68	18,092.46	1.52×
Decode-heavy output tok/s	74.37	1,647.90	1,645.66	22.13×
Mixed output tok/s	71.70	996.70	1,003.64	14.00×

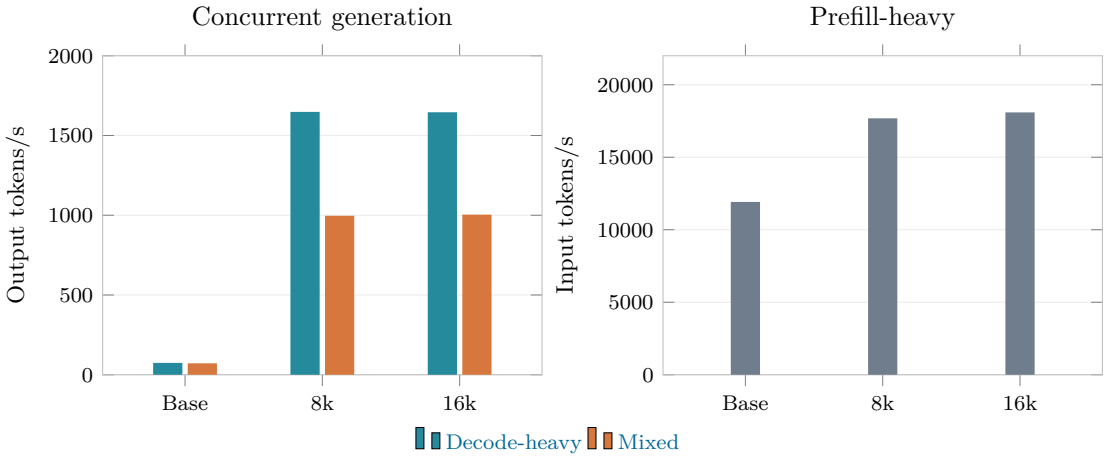


Figure 1: Matched concurrency-32 measurements. Prefill-heavy requests produce one output token; their rate includes scheduling and first-token generation, and is not isolated GPU prefill speed.

3.1 Batching dominates the first optimization step

The baseline can execute only one sequence at a time. Enlarging engine capacity allows concurrent requests to share execution steps, raising decode-heavy output throughput by 22.13×. Mixed throughput improves by 14.00×. The prefill-heavy gain is smaller (51.79%), consistent with a workload that already presents substantial token-level parallelism. This interpretation follows the configuration contrast; kernel-level causal attribution would require the cancelled profiles.

3.2 The larger token budget gives a modest incremental gain

Moving from 8,192 to 16,384 increases prefill input throughput by 2.31% and mixed output throughput by 0.70%, while decode-heavy throughput changes by -0.14%. These differences are small compared with enabling concurrency. No single budget is uniformly superior: the 8,192 setting has lower median mixed TTFT and lower observed memory use, while 16,384 has higher aggregate prefill throughput.

For the two measured repetitions, the largest relative output-rate spread across the four cases is 0.09% for 8,192 and 0.56% for 16,384, calculated as $(\max - \min)/\text{mean}$. We do not interpret these two-run ranges as uncertainty bounds on production workloads.

4 Latency and memory tradeoffs

Table 4: Client-observed latency: p50 / p95. Values are milliseconds.

Workload	Metric	Baseline	8,192 budget	16,384 budget
Single request	TTFT	62.89 / 63.91	98.54 / 100.09	98.00 / 98.75
Single request	TPOT	13.07 / 13.07	13.06 / 13.06	13.13 / 13.14
Prefill-heavy	TTFT	10,978.87 / 11,010.75	7,329.12 / 7,536.07	7,065.13 / 7,997.89
Decode-heavy	TTFT	53,413.02 / 53,425.81	335.37 / 339.57	333.70 / 335.75
Decode-heavy	TPOT	13.06 / 13.06	16.92 / 16.94	16.96 / 16.99
Mixed	TTFT	55,448.50 / 55,475.93	1,519.94 / 1,899.09	1,779.62 / 1,856.21
Mixed	TPOT	13.09 / 13.10	20.26 / 26.28	18.11 / 21.45

Throughput versus token latency. Decode-heavy TPOT p50 increases from 13.06 to 16.96 ms. Each concurrent request therefore receives tokens more slowly than a request already executing alone. Aggregate output rate nevertheless increases sharply because many requests make progress together. Baseline median decode TTFT is approximately 53.4 s because requests queue; the 16,384 candidate reduces it to approximately 0.334 s. These finite closed-loop experiments include startup/drain effects and are not open-loop service-level capacity tests.

A low-load regression. At concurrency one, median TTFT rises from 62.89 to 98.00 ms and output throughput declines by 2.45%. The graph capture policies differ, so this result does not isolate its cause. A throughput-oriented policy should not be presented as a universal latency improvement.

Table 5: Resource cost. GPU memory is the maximum of end-of-case snapshots, not a continuously sampled peak. All runs use the same 700 W power limit.

Resource	Baseline	8,192 budget	16,384 budget
Configured cache (GiB)	2	16	16
Observed GPU memory (GiB)	33.36	48.81	50.97
Prefill end-of-case power (W), range	680.5–685.1	678.0–681.1	681.3–685.0

The larger cache and working set reduce headroom for other model workers. The 16,384 candidate reaches 50.97 GiB in sampled GPU memory, compared with 33.36 GiB for the baseline. Co-resident model capacity must be re-evaluated before adoption. Prefill samples approach the power limit; clocks were not locked and varied with load. Power snapshots do not establish energy per request.

4.1 Compilation and CUDA graphs

Kernel compilation and CUDA graphs solve complementary problems: compilation optimizes generated GPU work, while graph replay reduces repeated launch overhead [5, 4]. Both were already enabled in the baseline. Cached compiled artifacts reduced observed compilation initialization from 58.45 s on an earlier startup attempt to 7.63 s in the successful baseline restart. This is a startup observation, not a steady-state throughput speedup or a controlled startup benchmark. All serving measurements exclude engine initialization and workload warmup. A graphs-off ablation was not performed.

5 Validity, quality and deployment boundary

5.1 What the results establish

The completed configurations passed three natural-language smoke checks (capital of France, 7×8, opposite of hot), completed every measured request at the required output length, and reported zero cached prompt tokens. The complete comparison uses unchanged checkpoint bytes and package versions. The benchmark's CPU tests cover wall-time throughput, cumulative streaming, single-token TPOT, concurrency bounds, incomplete output, timeout evidence and process cleanup; 16 tests passed locally.

These checks do not establish numerical equivalence or production model quality. Even the smoke outputs differ in capitalization ("cold" versus "Cold"). Synthetic token sequences are not meaningful quality examples. No image/video, structured-output, long-conversation or task-accuracy evaluation was completed. Backend arithmetic, batch shape and graph padding can affect generated tokens; the report does not claim identical outputs.

5.2 Threats to validity

- **Traffic model.** Only fixed lengths and concurrency 1/32 were measured fully. There is no variable-length arrival distribution, sustained overload, admission-rate sweep or production network path.
- **Attribution.** The main comparison jointly changes cache size, sequence capacity and graph policy. It cannot isolate batching, cache or graphs as the sole source of speedup.
- **Small repetition count.** Two repetitions per case provide limited evidence about stability. Trials were not randomized or interleaved, and the successful baseline rerun occurred after the two larger-budget candidates.
- **Timing semantics.** TTFT includes queueing; TPOT is based on streamed observations. Grouped first chunks can obscure true token-generation timing. GPU kernel timings and a kernel-level bottleneck breakdown are unavailable.
- **Resources.** Memory and power are point samples. The model manager remained active, but no other GPU compute process was present at experiment startup. No concurrent co-residency test or energy measurement was performed.

5.3 Final machine state

No remaining live lab processes were found. A subsequent GPU query returned no compute processes, while the production inference service remained active and untouched. The experiment folder, logs and raw results were copied to a local archive. No production configuration or model checkpoint was changed.

6 Recommendation

For the measured concurrency-32 workloads, retain the 16,384-token candidate as the throughput-oriented reference and 8,192 as the lower-memory, lower-median- mixed-TTFT alternative. Neither is a validated production default. To expose the measured engine capacity to users, a future integration must replace the single-request busy lock with asynchronous admission and bounded backpressure, preserve cancellation and structured/multimodal behavior, and update memory accounting for co-resident models.

The next kernel step is profiling after warmup, followed by one backend or kernel ablation with a numerical reference and matched end-to-end tests [5]. Compilation and graph capture should remain part of that procedure. Current evidence supports a substantial engine-level throughput gain; it does not yet support a claim of custom-kernel acceleration or saturation of the H100.

A Reproduction and artifact map

Archive root: this repository.
GPU workspace: a temporary lab directory on the H100 host, removed after hash-verified transfer. Inference is stopped. Generating this paper and analyzing its data require no GPU.

Table 6: Source evidence for the reported configurations.

Configuration	Raw JSON under `results/`
Baseline	`20261001T113011.178231Z/baseline.json`
8,192-token candidate	`20261001T102025.566538Z/batch64_t8192.json`
16,384-token candidate	`20261001T102025.566538Z/batch64_t16384.json`
Interrupted candidate	`20261001T113011.178231Z/batch64_t2048.json`

Each result stores per-request token counts and timings, output-token hashes, workload definitions, package versions, hardware snapshots, model path, resolved engine configuration and source hashes. Initialization logs preserve selected backends, compile/cache messages and GPU-cache sizing. Two initial attempts failed because `ninja` was absent from the launch PATH; the installed Python binary directory was added without changing dependencies. The broad cancellation marker also tagged these earlier incomplete JSON files as cancelled; their nonzero exit codes and original failure logs identify them as startup failures. They are excluded throughout.

Build this paper locally:

```
python3 paper/build_data.py
cd paper
tectonic paper.tex
```

Benchmark entry points, retained for a later authorized GPU run:

```
python run_suite.py --configs baseline batch64_t8192 batch64_t16384
python run_suite.py --configs batch128_t16384 \
  --workloads workloads_scale.json
python run_suite.py --configs batch64_t16384 \
  --workloads workloads_profile.json --repetitions 1 --profile
```

On the H100 host, `python` above means the pinned Python 3.14.6 installation. `run_suite.py` acquires a lab lock, checks GPU idleness and serializes runs. Do not execute the retained queue script merely to rebuild this paper.

References

[1] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023. https://arxiv.org/abs/2309.06180.

[2] Qwen Team. *Qwen3.8-27B model card*, 2026. https://huggingface.co/Qwen/Qwen3.8-27B.

[3] vLLM Project. *Optimization and Tuning*. https://docs.vllm.ai/en/latest/configuration/optimization/.

[4] vLLM Project. *CUDA Graphs* and *torch.compile integration*. https://docs.vllm.ai/en/latest/design/cuda_graphs/; https://docs.vllm.ai/en/latest/design/torch_compile/.

[5] NVIDIA. *CUDA C++ Best Practices Guide*. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/.

Online references consulted on 1 October 2026. Installed runtime logs, rather than newer online defaults, define the configurations measured here.